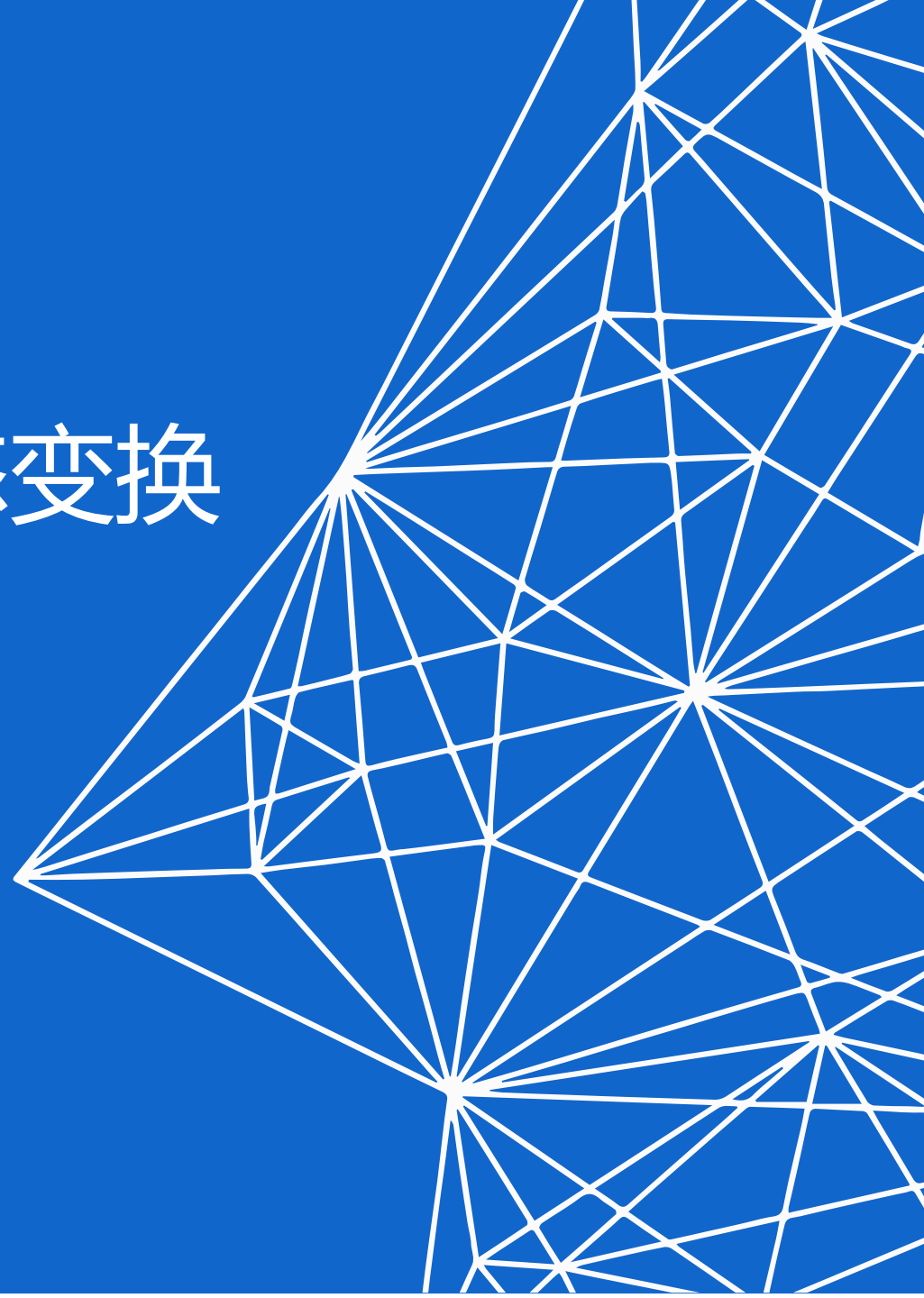




第五节：图像滤波与形态变换

杨聪





目录

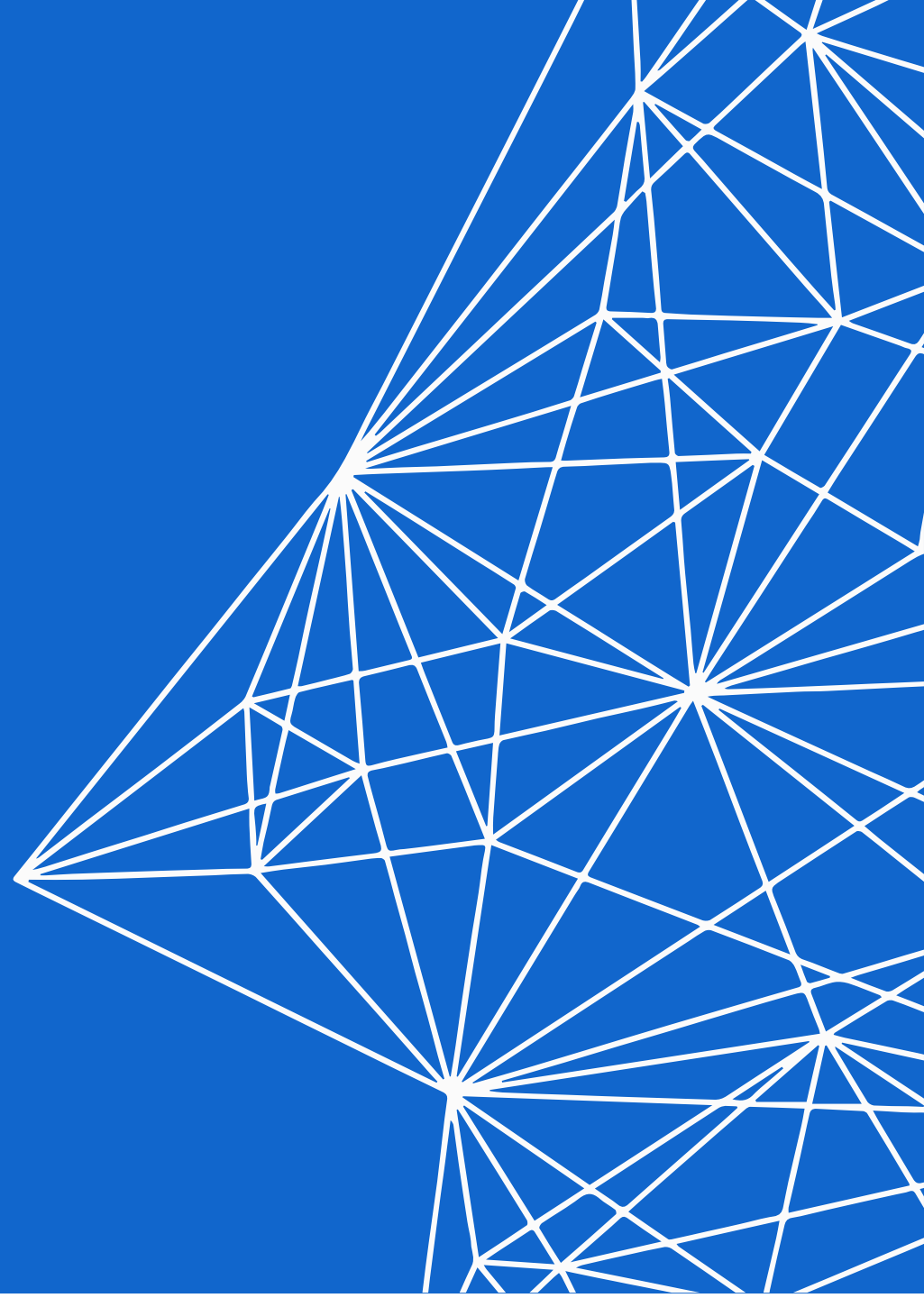
CONTENTS

1. 图像模糊

- 均值滤波
- 高斯滤波
- 方框滤波
- 中值滤波
- 双边滤波
- 2D卷积

2. 形态操作

- 膨胀
- 腐蚀
- 骨架



为什么图像是波

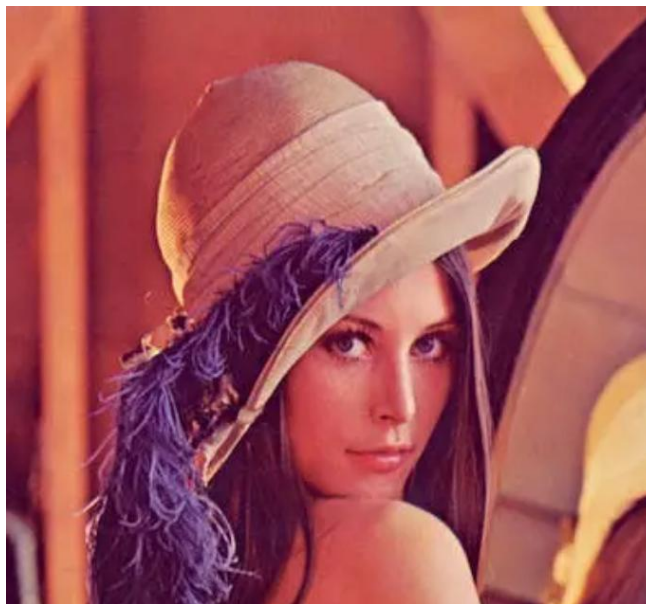
我们知道，**图像**在计算机中是以离散形式存储和表示的。通常用一个三维张量 $H \times W \times C$ 来表示一幅图像。

而**波**（以波函数 $y(x)$ 表示）是指 y 随 x 变化而产生的变化轨迹，在 xy 平面上是一段连续的曲线。

那么，图像和波该如何联系起来呢？

答案是“**降维**”。

举例来说，左边是一张分辨率为 400×400 的图片，一共包含了 16 万个像素点。



为什么图像是波

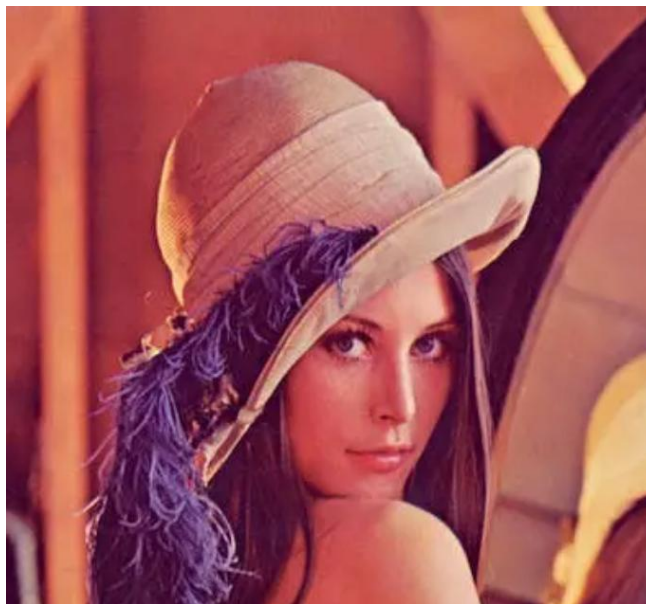
我们知道，**图像**在计算机中是以离散形式存储和表示的。通常用一个三维张量 $H \times W \times C$ 来表示一幅图像。

而**波**（以波函数 $y(x)$ 表示）是指 y 随 x 变化而产生的变化轨迹，在 xy 平面上是一段连续的曲线。

那么，图像和波该如何联系起来呢？

答案是“**降维**”。

举例来说，左边是一张分辨率为 400×400 的图片，一共包含了 16 万个像素点。

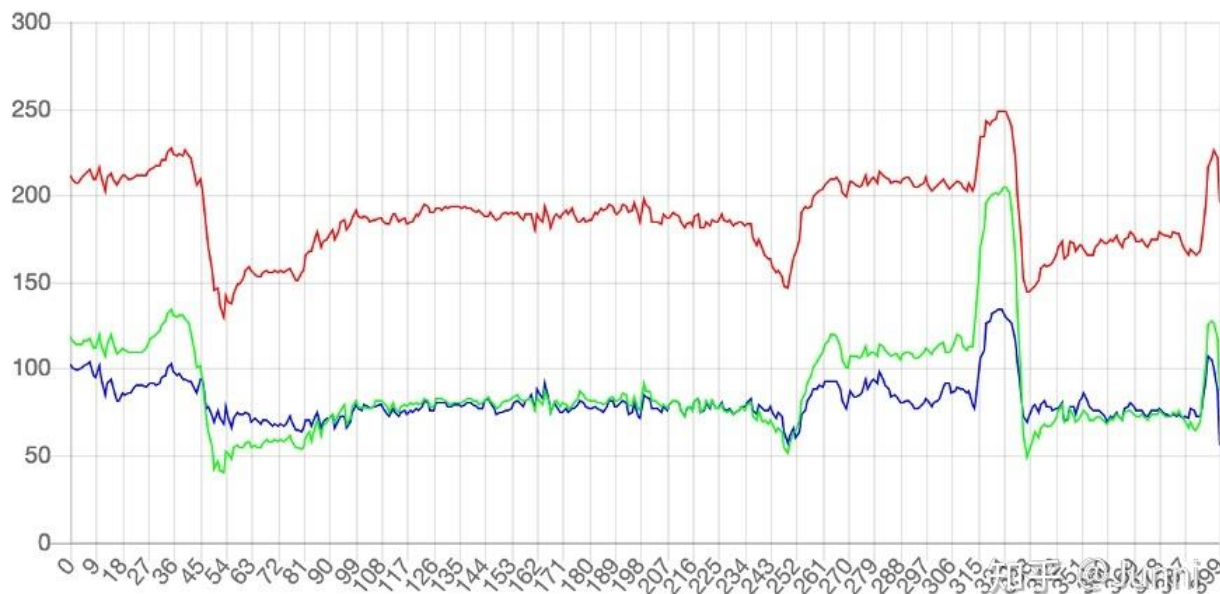


图像模糊

为什么图像是波

每个像素的颜色，可以用红、绿、蓝、透明度四个值描述，大小范围都是 $0 \sim 255$
比如黑色是 $[0, 0, 0, 255]$ ，白色是 $[255, 255, 255, 255]$

如果把**每一行**所有像素（上例是 400 个）的红、绿、蓝的值，依次画成三条曲线，
就得到了下面的波形：



可以看到，每条曲线都在不停的上下波动。有些区域的波动比较小，有些区域突然出现了大幅波动（比如 54 和 324 这两点）。对比一下图像就能发现，曲线波动较大的地方，也是图像出现突变的地方。

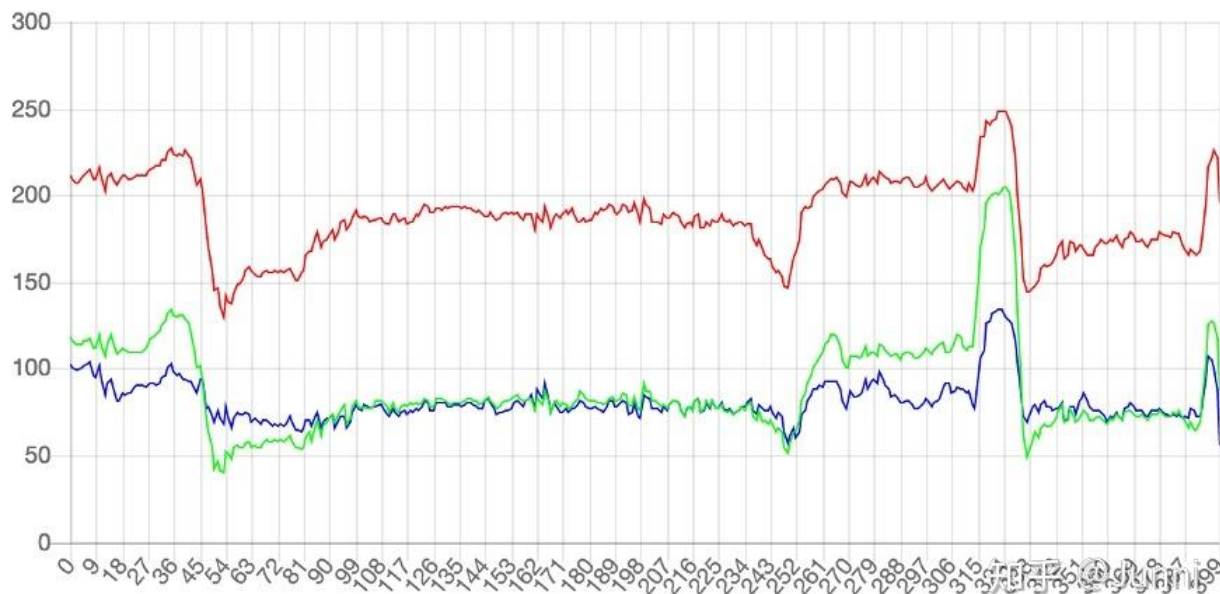


图像模糊

为什么图像是波

每个像素的颜色，可以用红、绿、蓝、透明度四个值描述，大小范围都是 $0 \sim 255$
比如黑色是 $[0, 0, 0, 255]$ ，白色是 $[255, 255, 255, 255]$

如果把**每一行**所有像素（上例是 400 个）的红、绿、蓝的值，依次画成三条曲线，
就得到了下面的波形：



这说明波动与图像是紧密关联的。图像本质上就是各种色彩波的叠加。





频率

介绍完图像与波的联系以及波的频率，下面介绍一种与频率有关的处理波的方法。

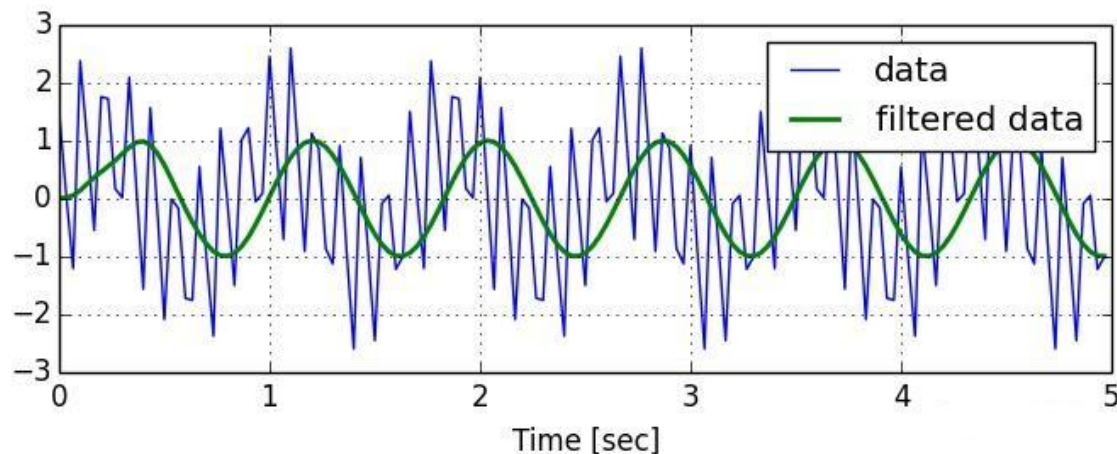
物理学对波的研究已经非常深入，提出了很多处理波的方法，其中就有滤波器（filter）：过滤掉某些波，保留另一些波。两种常见的滤波器是：

- 低通滤波器（lowpass）：减弱或阻隔高频信号，保留低频信号。
- 高通滤波器（highpass）：减弱或阻隔低频信号，保留高频信号。

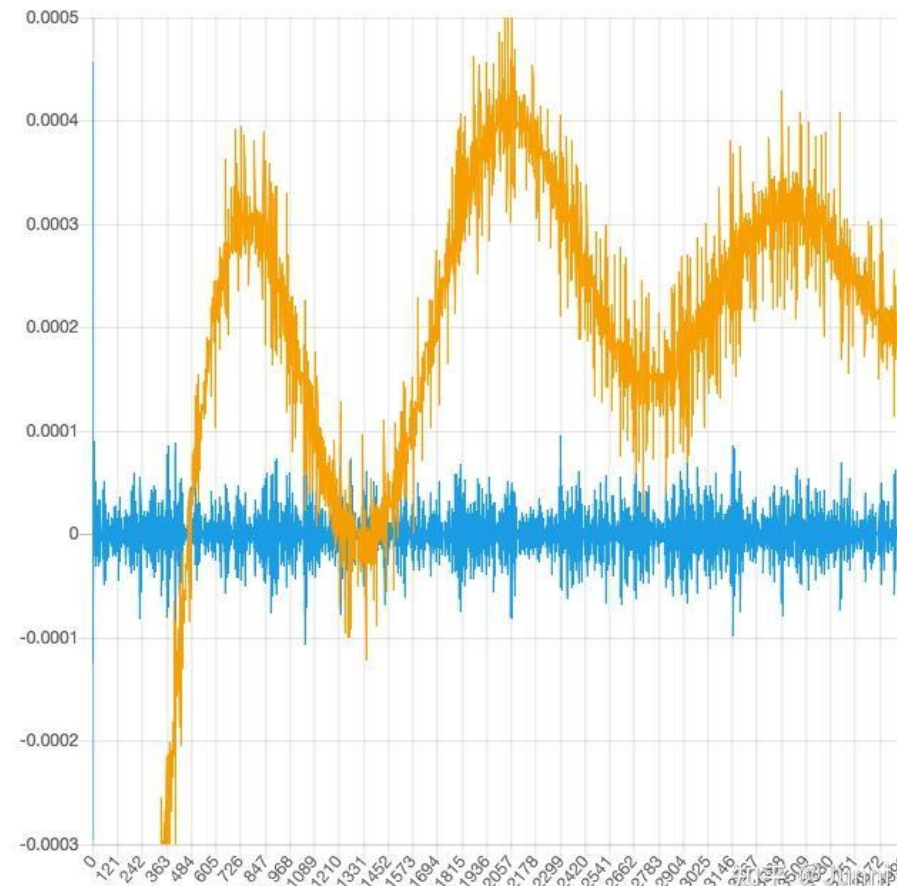
下面分别举例说明两种滤波器提取得到的波形，通过图形可以直观地感受到两种滤波器在做什么。

图像模糊

频率



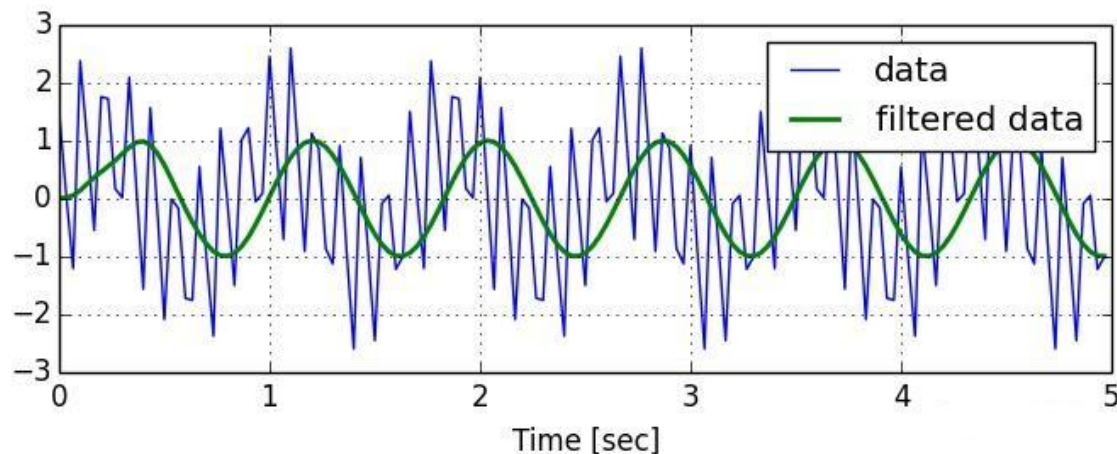
蓝线是原始的波形，绿线是低通滤波后的波形。
可以看到，绿线的波动比蓝线小很多，非常平滑。



黄线是原始的波形，蓝线是高通滤波后的波形。可以看到，黄线的三个波峰和两个波谷（低频波动），在蓝线上都消失了，而黄线上那些密集的小幅波动（高频波动），则是全部被蓝线保留。

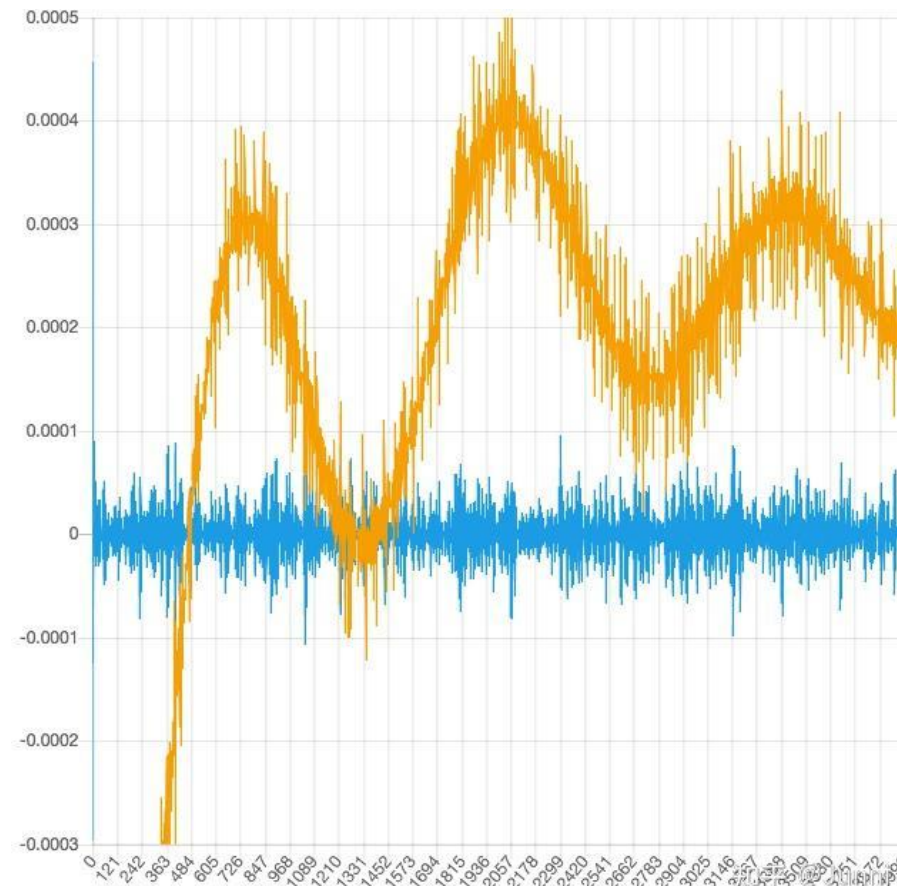
图像模糊

频率



蓝线是原始的波形，绿线是低通滤波后的波形。
可以看到，绿线的波动比蓝线小很多，非常平滑。

直观地说，低通滤波保留了波形整体上的低频波动，而高通滤波保留了波形局部上的高频波动。



黄线是原始的波形，蓝线是高通滤波后的波形。可以看到，黄线的三个波峰和两个波谷（低频波动），在蓝线上都消失了，而黄线上那些密集的小幅波动（高频波动），则是全部被蓝线保留。

图像的滤波

正如前文所述，图像“降维”后可以用波来表示，因此可以将滤波器应用于图像处理。

下面分别用低通滤波器和高通滤波器对图像进行滤波，并将结果可视化。

低通滤波器保留图像的低频信息，过滤掉高频信息，使得色彩变化剧烈的区域变得平滑，**也就是出现模糊效果。**



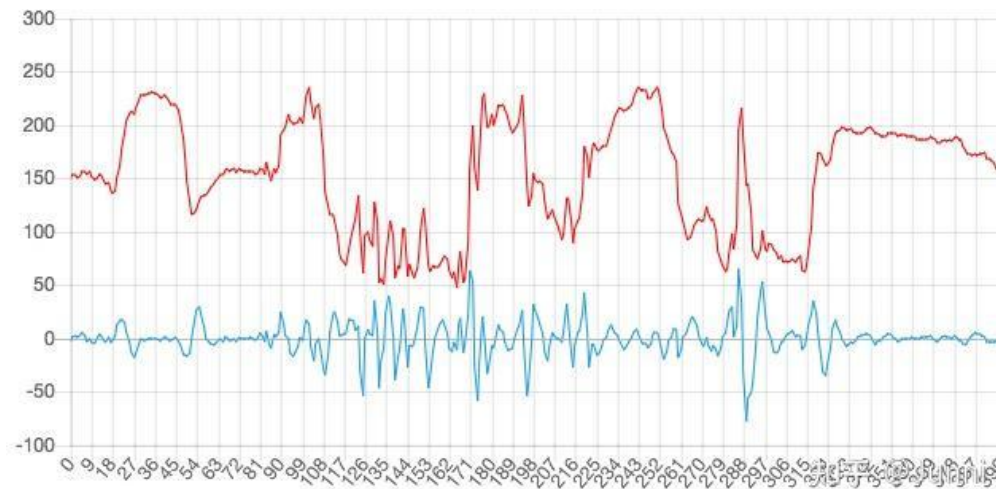
上图中，红线是原始的色彩曲线，蓝线是低通滤波后的曲线。

图像的滤波

正如前文所述，图像“降维”后可以用波来表示，因此可以将滤波器应用于图像处理。

下面分别用低通滤波器和高通滤波器对图像进行滤波，并将结果可视化。

高通滤波器正好相反，过滤了低频，只保留那些变化最快速最剧烈的区域，**也就是图像里面的物体边缘，所以也常用于边缘检测。**



上图中，红线是原始的色彩曲线，蓝线是高通滤波后的曲线。

图像模糊

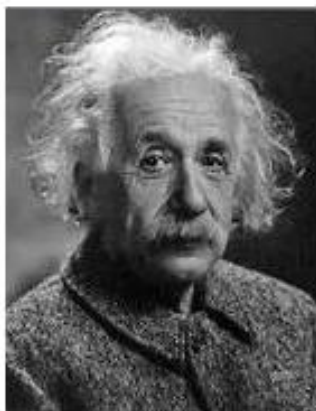


Fig1. 原图



Fig2. 均值滤波



Fig3. 高斯滤波

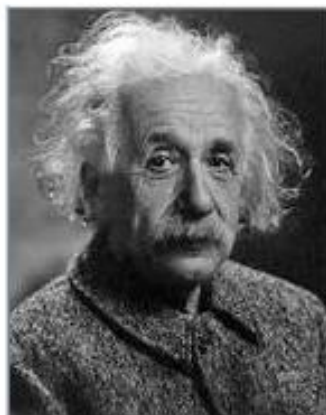


Fig4. 盒子滤波



Fig5. 中值滤波

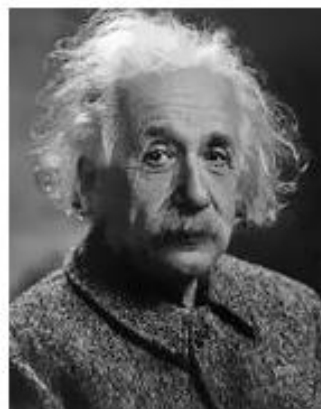
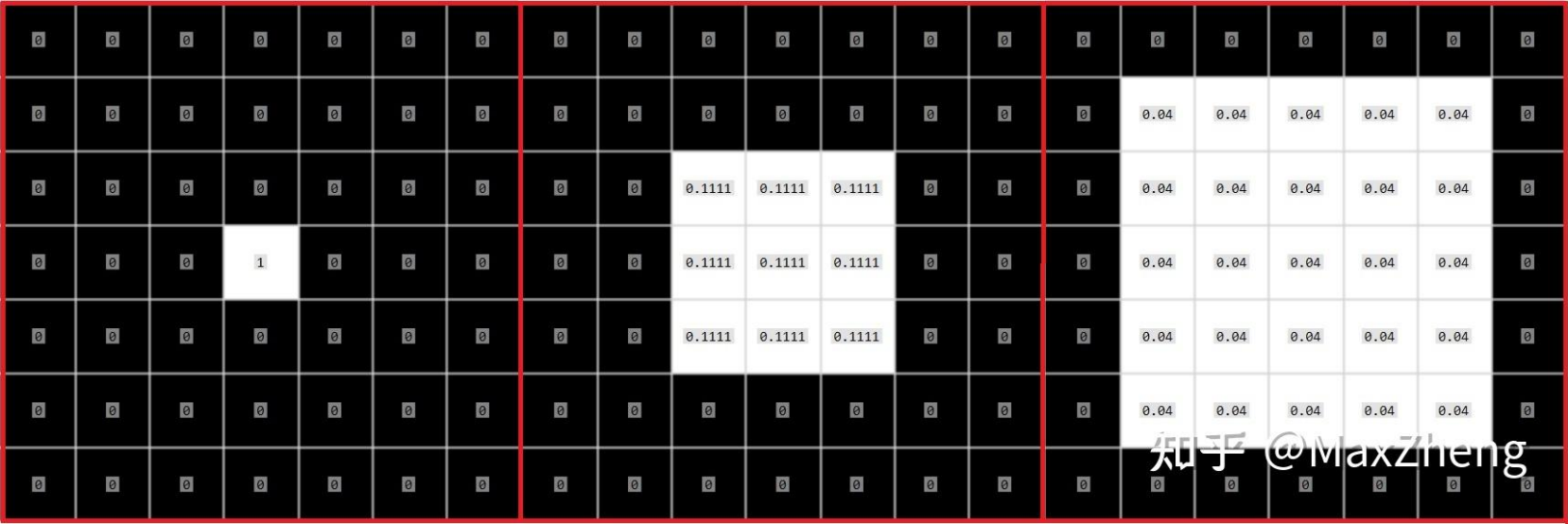


Fig6. 双边滤波

图像滤波是图像处理的最基本手段之一，如常见的美颜、去噪、马赛克等，其背后核心技术就是滤波。

均值滤波

均值滤波的原理就计算一个像素点及其周围一定范围内的点的平均值，再重新赋值给该点，就达到了均值滤波的效果。如下图展示，从左值右一次为原图、半径为1（窗口大小3*3）的均值滤波结果、半径为2（窗口大小5*5）的结果。



根据其原理，均值滤波的直观实现很简单，就是遍历图像。再对每一个像素点遍历其周围一定半径的像素，进行均值求解即可



高斯滤波

上一页展示的均值滤波，其窗口内所有的像素的加权平均的权重相等。而我们在实际应用的时候，往往想要距离越远的像素，其模糊权重越小，高斯模糊完美解决了这个问题。

高斯模糊是一种图像模糊滤波器，它用正态分布计算图像中每个像素的变换。在二维空间正态分布方程为：

$$G(u, v) = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/(2\sigma^2)}$$

σ 是正态分布的标准偏差。

在二维空间中，这个公式生成的曲面的等高线是从中心开始呈正态分布的同心圆。分布不为零的像素组成的卷积矩阵与原始图像做变换。每个像素的值都是周围相邻像素值的加权平均。原始像素的值有最大的高斯分布值，所以有最大的权重，**相邻像素随着距离原始像素越来越远，其权重也越来越小**。这样进行模糊处理比其它的均衡模糊滤波器更高地保留了边缘效果。

高斯滤波

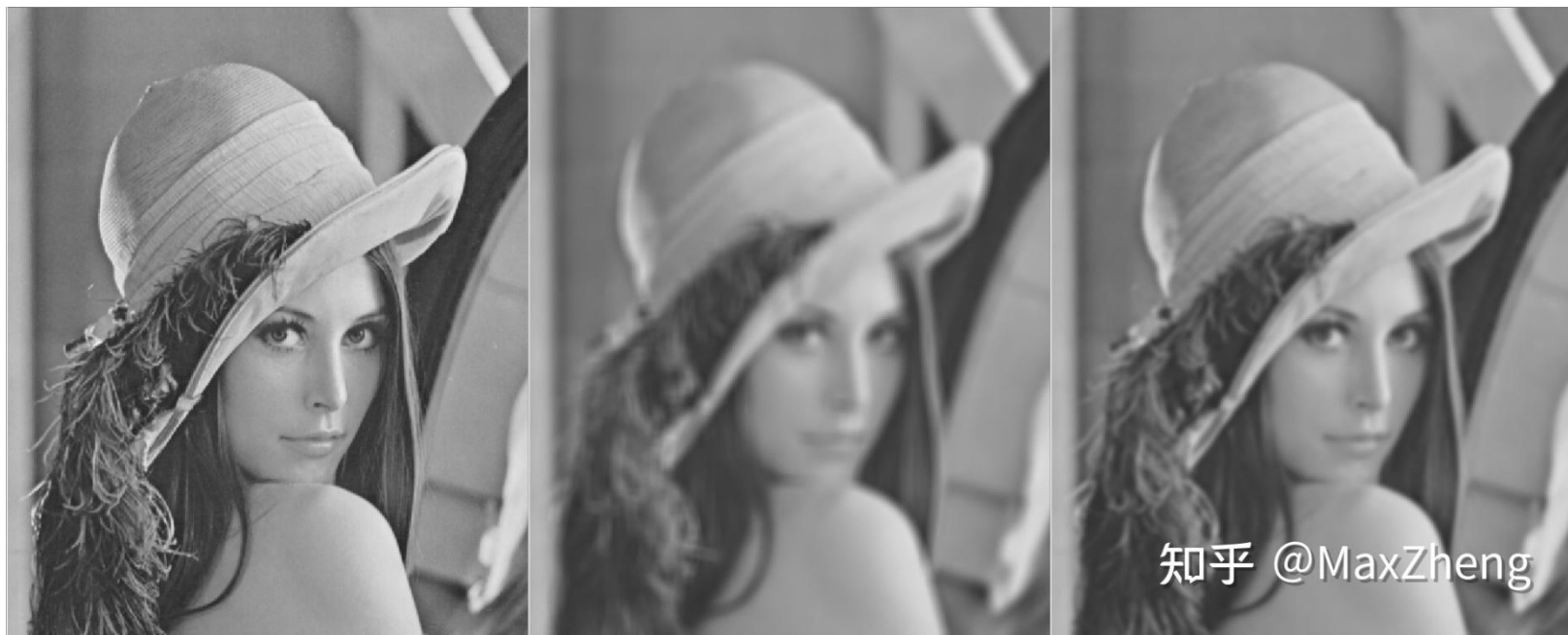
上一页展示的均值滤波，其窗口内所有的像素的加权平均的权重相等。而我们在实际应用的时候，往往想要距离越远的像素，其模糊权重越小，高斯模糊完美解决了这个问题。



半径为2，标准差为0时的二维高斯权重分布

高斯滤波

上一页展示的均值滤波，其窗口内所有的像素的加权平均的权重相等。而我们在实际应用的时候，往往想要距离越远的像素，其模糊权重越小，高斯模糊完美解决了这个问题。



原图

半径5的均值滤波

半径5、标准差0的高斯滤波

可以看出，高斯模糊和均值模糊的唯一区别就是高斯模糊添加了空间权重的影响，因此高斯模糊的边缘更加锐利。

方框滤波

方框滤波和均值滤波核基本上是一致的，主要的区别是要不要归一化处理，如果使用归一化处理，方框滤波就是均值滤波。如果选择进行归一化，则滤波结果为邻域内点的像素值之和的平均值，否则滤波结果为像素值之和。



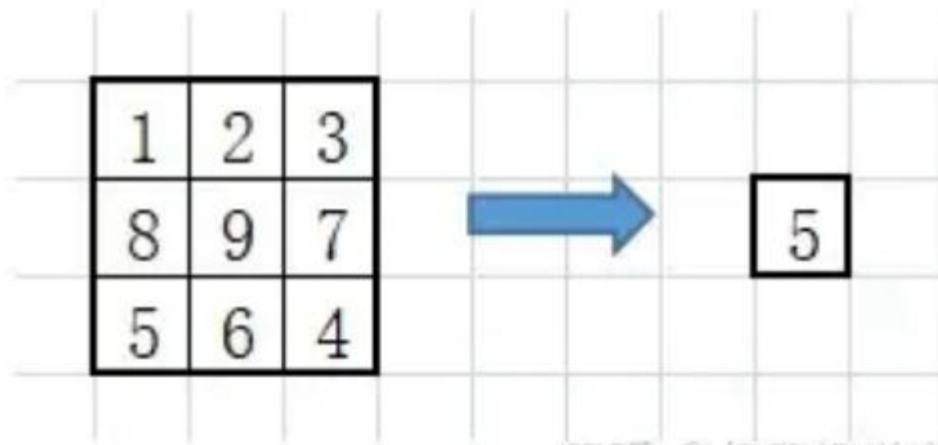
归一化



未归一化

中值滤波

中值滤波将邻域内的所有像素值排序，取中间值作为邻域中心点的像素值，因此中值滤波又叫平滑处理，因为这个方法**既能抑制噪声又能保持细节**。



它将每一像素点的灰度值设置为该点某邻域窗口内的所有像素点灰度值的中值



双边滤波

最原始的去噪方法是均值滤波（或高斯均值滤波）。对图像中一个像素点 x ，设定一个大小一定的方形邻域，用邻域内其他像素的灰度值的（加权）平均值来更新这个点的像素。加权的方式一般是离 x 越近，则权重越大。很显然，这种“平均”的手段目的就是平滑图像，减少噪声点的突兀。不过这种无差别处理手段带来的负面影响就是将图像中的细节也模糊了。为了能够在去噪的同时保持细节信息，双边滤波诞生了。

双边滤波在计算像素值的同时会考虑距离和色差信息，从而可在消除噪声的同时保护边缘信息。在执行双边滤波操作时，如果像素点与当前点色差较小，则赋予其较大的权重值，否则赋予其较小的权重值。

一句话总结：邻域滤波时的加权系数不仅仅考虑几何距离，而且考虑灰度相似性。



双边滤波

双边滤波在计算像素值的同时会考虑距离和色差信息，从而可在消除噪声的同时保护边缘信息。在执行双边滤波操作时，如果像素点与当前点色差较小，则赋予其较大的权重值，否则赋予其较小的权重值。

一句话总结：邻域滤波时的加权系数不仅仅考虑几何距离，而且考虑灰度相似性。

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi$$

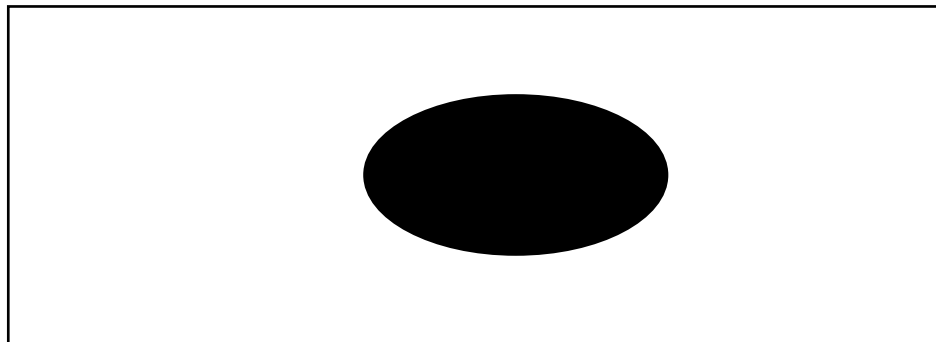
这里 $\mathbf{f}$ 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。

$$c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

双边滤波

原理：

对于噪声和细节我们可以有一个初步的区分认识：那就是如果在一个平滑区域出现一个噪点，噪声附近的点与它灰度值差距是差不多大的。例如在一个白色区域，出现了一个黑点噪声，那么邻域中的正常点因为与它灰度差距差不多进而分配到的相似度权值基本一样。这样，滤波便近似于高斯均值滤波。但是对于边缘细节部分，我们本身的要求就是想让灰度值差距大的一边（即边界外部）分配到的权重更小，而边界内部分配到的权重大，这恰好就是相似度函数 s 能够提供的。所以，这样一个直观的认识就能让我们清楚双边滤波为什么可以保持边界细节信息。





双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像, $\mathbf{h}$ 是恢复后的图像, c, s 分别是度量几何距离与灰度相似性的函数, k 是一个标准化系数。具体来说, 一般取高斯核函数, 分别依赖于两个标准差参数 σ_d, σ_r , 合适的大小可以由实验分析获得。

首先, 我们抛开函数 c , 单独分析一下函数 s 起了一个什么作用。

如果不带上 c , 利用 s 函数决定的权值来进行全局的滤波 (即不设置邻域, 每个点都提供权值):

$$\mathbf{h}(\mathbf{x}) = k_r^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi$$

那么其实这样的功能仅仅是将图像的灰度直方图向波峰挤压 (也就是变窄了)。因为上述公式我们可以转换为:

$$h(x) = \int_0^{\infty} \phi \tau(\phi, f(x)) d\phi$$



双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。

$$\mathbf{h}(\mathbf{x}) = k_r^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi$$

那么其实这样的功能仅仅是将图像的灰度直方图向波峰挤压（也就是变窄了）。因为上述公式我们可以转换为：

$$h(x) = \int_0^{\infty} \phi \tau(\phi, f(x)) d\phi$$

这里 ϕ 是某一个灰度值， $\nu(\phi)$ 是关于灰度值的直方图统计函数（在连续情况下可以理解为二维图像中灰度值为 ϕ 的点集测度）。上述转换的严格数学过程在此不作证明，本质可以看作是从黎曼积分转换成了勒贝格积分。

$$\tau(\phi, f) = \frac{s(\phi, f) \nu(\phi)}{\int_0^{\infty} s(\phi, f) \nu(\phi) d\phi}$$



双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。

$$\mathbf{h}(\mathbf{x}) = k_r^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi$$

那么其实这样的功能仅仅是将图像的灰度直方图向波峰挤压（也就是变窄了）。因为上述公式我们可以转换为：

$$h(x) = \int_0^{\infty} \phi \tau(\phi, f(x)) d\phi$$

这里 ϕ 是某一个灰度值， $v(\phi)$ 是关于灰度值的直方图统计函数（在连续情况下可以理解为二维图像中灰度值为 ϕ 的点集测度）。上述转换的严格数学过程在此不作证明，本质可以看作是从黎曼积分转换成了勒贝格积分。

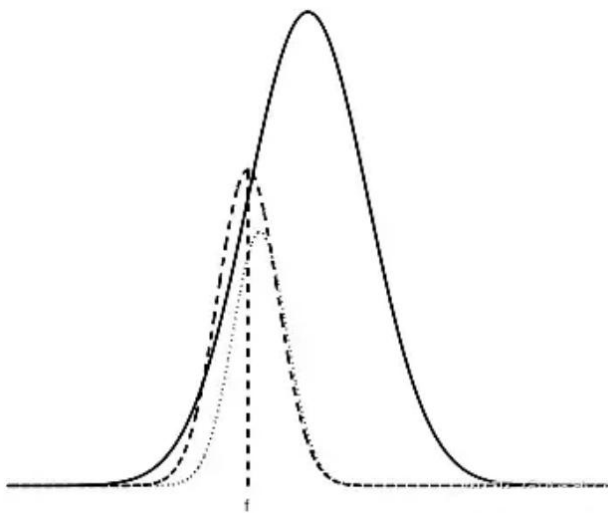
$$\tau(\phi, f) = \frac{s(\phi, f) v(\phi)}{\int_0^{\infty} s(\phi, f) v(\phi) d\phi}$$

双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。

假设下图是单峰图像的灰度直方图曲线（实线部分）：

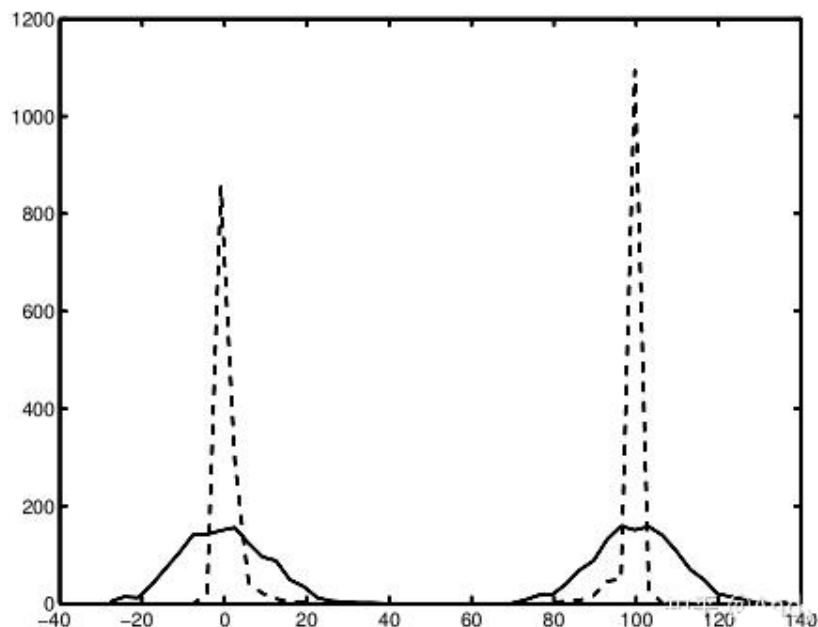


对于峰值左边的一点灰度值，由于灰度值大于它的像素更多，所以这样做滤波后它会向右移。同理，对于峰值右边的，会向左移，从而造成曲线向中间挤压。初步看来，这样一种滤波似乎只是一种直方图变换，并无大用。但当他与邻域均值滤波结合在一起时便大有可为，因为这时，这种变换只是在一个小小的窗口内进行。

双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。



左图是边界处一个窗口中的灰度曲线（实线部分），左右两边分别对应着黑色部分和白色部分（即边界内外）。当实行双边滤波后，他的灰度曲线变为虚线部分。可以看出，这样的变换将使得边界更加分明。

双边滤波

$$\mathbf{h}(\mathbf{x}) = k^{-1}(\mathbf{x}) \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \mathbf{f}(\xi) c(\xi, \mathbf{x}) s(\mathbf{f}(\xi), \mathbf{f}(\mathbf{x})) d\xi \quad c(\xi, x) = e^{-(\xi-x)^2/2\sigma_d^2} \quad s(f(\xi), f(x)) = e^{-(f(\xi)-f(x))^2/2\sigma_r^2}$$

这里 f 是含噪图像， $\mathbf{h}$ 是恢复后的图像， c, s 分别是度量几何距离与灰度相似性的函数， k 是一个标准化系数。具体来说，一般取高斯核函数，分别依赖于两个标准差参数 σ_d, σ_r ，合适的大小可以由实验分析获得。



原图

带噪图像

双边滤波结果

上图是实验效果，可以看出边界处保持的较好，但去噪效果一般。事实上，双边滤波是二十年前方法，对比BM3D， Sparse Coding等SOTA方法已经相当过时，但它提供的想法是非常值得借鉴的。

2D卷积

自定义的卷积核来执行滤波操作：利用卷积核在图像上滑动，将图像点上的像素值与对应的卷积核上的数值相乘，然后将所有相乘后的值相加作为卷积核中间像素点的像素值，并最终滑动完所有图像的过程。

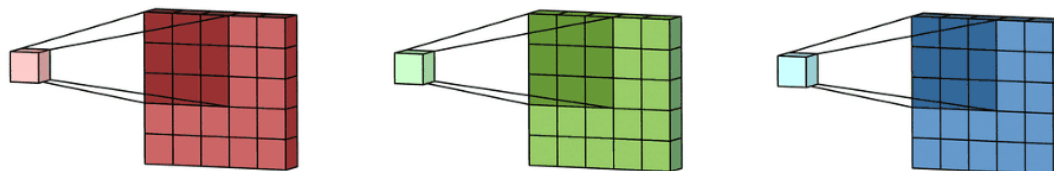
单通道卷积

stride = 1
padding = 0
kernel =
[[0,1,2],
[2,2,0],
[0,1,2]]

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

2D卷积

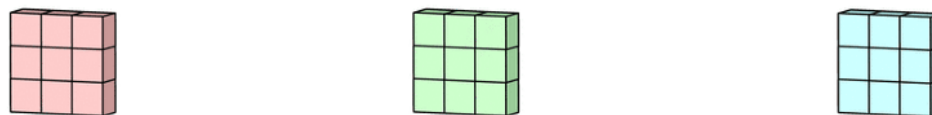


多通道卷积

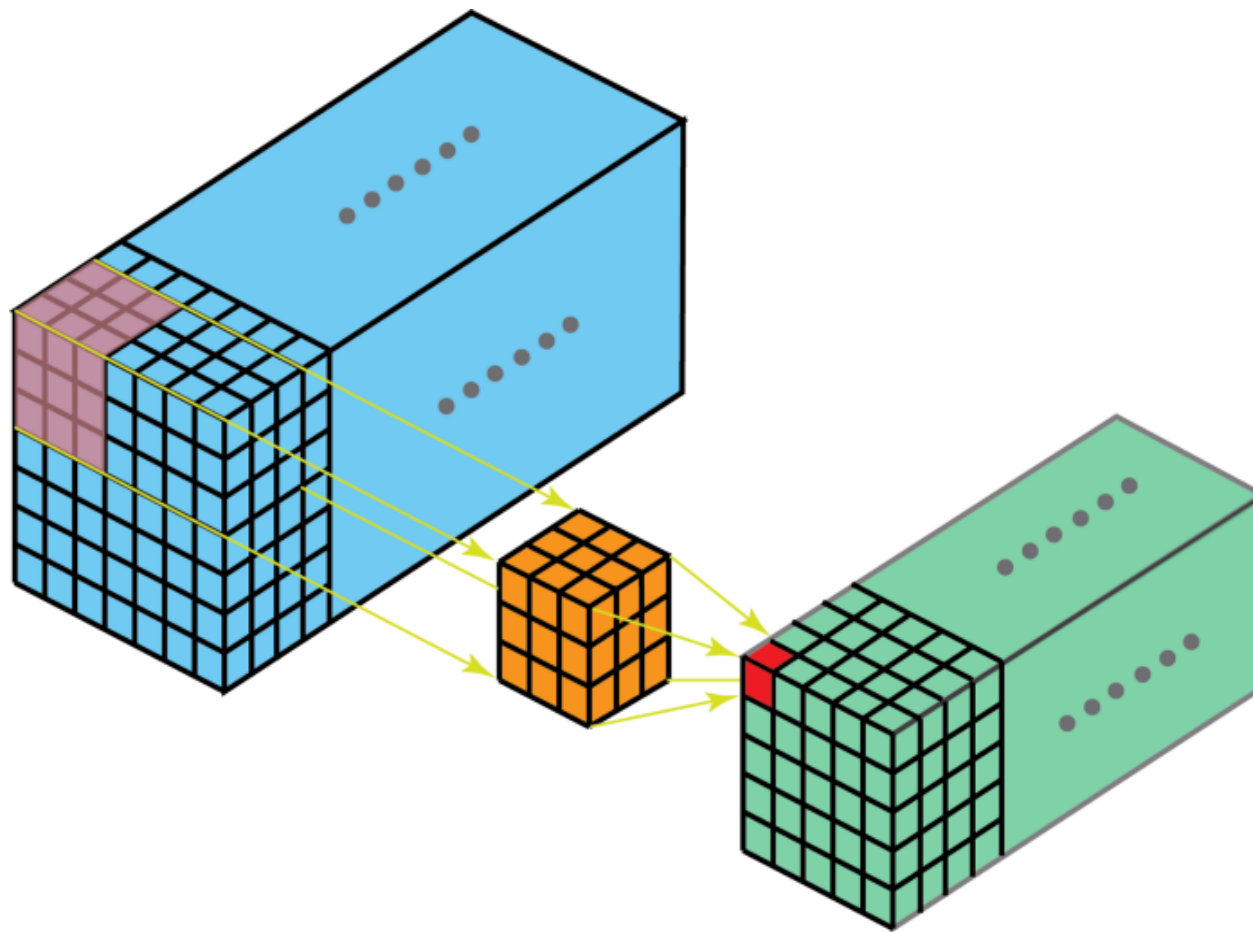
stride = 1

padding = 0

kernel size = 3x3x3



3D卷积





思考题

1. 输入图像大小 (Input Size)，卷积核 (Kernel Size)，以及输出图像大小 (Output Size) 之间的数学关系？
2. 如果考虑Stride与Padding的值，上述三者之间的关系？
3. 你能用Python写一个2D卷积吗？



图像模糊

方法	原理	优点	缺点
均值滤波	线性滤波，通过求窗口内所有像素的平均值来得到中心像素点的像素值。	效率高，思路简单；有效的平滑图像，降低图像的尖锐程度，降低噪声。	不能消除噪声；计算均值会将图像中的边缘信息以及特征信息“模糊”掉，会丢失很多特征。
高斯滤波	线性滤波，是常用的一种滤波算法，利用二维高斯函数的分布方式来对图像进行平滑。	很好地滤除高斯噪声，而且对图像的平滑更好，不会造成图像的大幅度模糊，对边缘的处理效果也较好。	只能运用于高斯噪声，对椒盐噪声等其它噪声很难达到好的滤波效果。
方框滤波	方框滤波和均值滤波核基本一致，区别是需不需要均一化处理。	速度快。	容易导致像素溢出
中值滤波	非线性滤波，通过找窗内的所有像素值的中值然后赋给中心像素点。	处理脉冲噪声以及椒盐噪声时效果极佳，能够有效的保护好图像的边缘信息。	由于需要进行排序操作，中值滤波的计算量较大。
双边滤波	非线性滤波，结合图像的空间邻近度和像素值相似度的一种折衷处理，同时考虑空域信息和灰度相似性	可平滑平坦区域，同时保持边缘锐化。	效率低，不能去除高频噪声和脉冲噪声
2D卷积	自定义的卷积核来执行滤波操作。利用卷积核在图像上滑动，将图像点上的像素值与对应的卷积核上的数值相乘，然后将所有相乘后的值相加作为卷积核中间像素点的像素值，并最终滑动完所有图像的过程。	灵活性强	对比其他方法较复杂

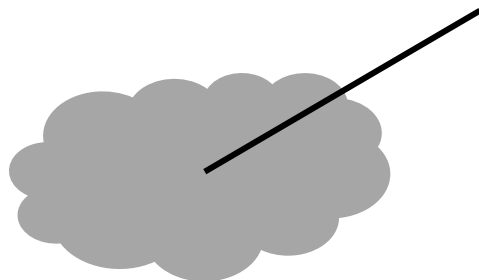


典型应用：边缘检测（Edge Detection）

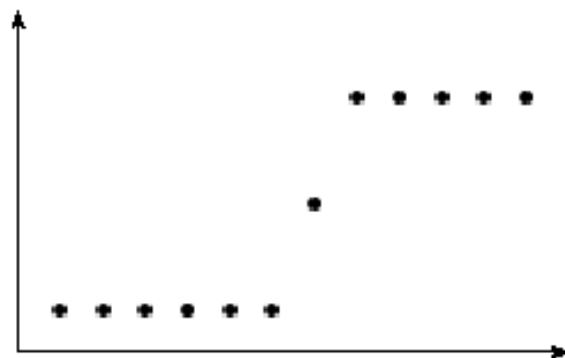
- 物体的边缘以图像局部特征不连续的形式出现，也就是指图像局部亮度变化最显著的部分，例如：灰度值的突变，颜色的突变和纹理结构的突变等，同时物体的边缘也是不同区域的分界处。
- 图像边缘有方向和幅度两个特性，**通常沿边缘走向的像素灰度变化平缓，垂直于边缘走向的像素灰度变化剧烈。**
- 边缘检测的实质是采用某种算法来提取出图像中对象与背景间的交界线。**图像灰度的变化情况可以用图像灰度分布的梯度来反映**，因此我们可以用局部图像微分技术来获得**边缘检测算子**。
- 经典的边缘检测方法，是通过对原始图像中像素的某小邻域构造边缘检测算子来达到检测边缘这一目的（**使用差分近似图像函数导数的算子**）。

在一个邻域中对像素的灰度平均会使图像中的细节变得模糊。
差分产生相反的效果，即锐化图像。

边缘具有斜坡的一维形状，计算图像的导数会突出显示其位置。

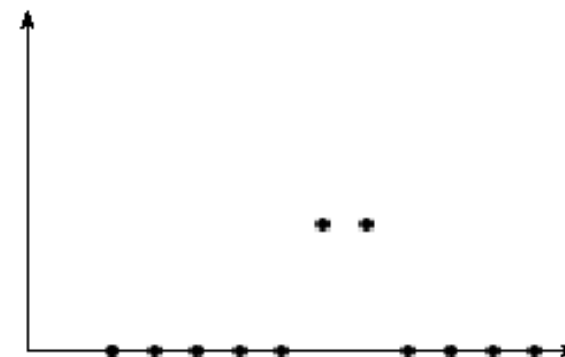


交叉边缘的直线



Grey values across edges
(a ramp edge)

斜坡



Difference of the
Edge function

- 差分（difference）可以突出边缘，而弱化其他部分
- 利用差分来得到边缘提取的效果



图像处理卷积核——算子

在对图像的操作，我们采用模板对原图像进行卷积运算，从而达到我们想要的效果。而获取一幅图像的梯度就转化为：**模板（Roberts、Prewitt、Sobel、Laplacian算子）对原图像进行卷积。**

- 在**一维**连续数集上有函数 $f(x)$ ，我们可以通过求导获得该函数在任一点的斜率，根据导数的定义有：

$$f'(x) = \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

- 在**二维**连续数集上有函数 $f(x,y)$ ，我们也可以通过求导获得该函数在x和y分量的偏导数，根据定义有：

$$\frac{\partial f(x, y)}{\partial x} = \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x}$$

$$\frac{\partial f(x, y)}{\partial y} = \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y}$$



梯度和Roberts算子

对于图像来说，是一个二维的离散型数集，通过推广二维连续型求函数偏导的方法，来求得图像的偏导数，即在(x,y)处的最大变化率，也就是这里的梯度：

$$g_x = \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

$$g_y = \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

梯度是一个矢量，则(x,y)处的梯度表示为：

$$\nabla f \equiv grad(f) \equiv [g_x, g_y]^T = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$$

其大小为：

$$M(x, y) = mag(\nabla f) = \sqrt{g_x^2 + g_y^2}$$

梯度和Roberts算子

平方和平方根需要大量的计算开销，所以使用绝对值来近似梯度幅值： $M(x, y) \approx |g_x| + |g_y|$

方向与 $\alpha(x, y)$ 正交： $\alpha(x, y) = \arctan\left[\frac{g_y}{g_x}\right]$

上右图是图像的垂直和水平梯度，但我们有时候也需要对角线方向的梯度，定义如下：

$$g_x = \frac{\partial f}{\partial x} = f(x+1, y+1) - f(x, y)$$

$$g_y = \frac{\partial f}{\partial y} = f(x+1, y) - f(x, y+1)$$

注：Roberts算子对噪声敏感

垂直和水平梯度的模板为：

$$g_x = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad g_y = \begin{bmatrix} -1 & 1 \end{bmatrix}$$

对角线梯度的模板为：

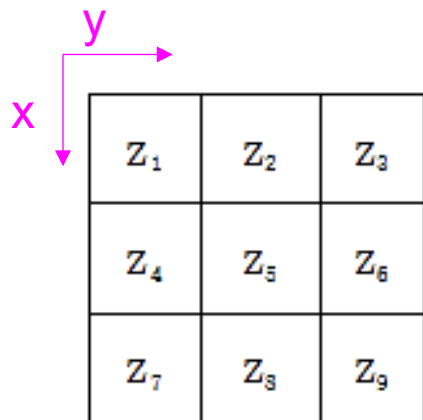
$$\begin{matrix} & y \\ x \downarrow & \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \end{matrix} \quad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$



Roberts算子

图像模糊

Prewitt 算子



3*3 模板

定义水平、垂直和两对角线方向的梯度：

- 水平方向：
$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$
- 垂直方向：
$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$
- 对角线方向：
$$g'_x = (z_2 + z_3 + z_6) - (z_4 + z_7 + z_8)$$

$$g'_y = (z_6 + z_8 + z_9) - (z_1 + z_2 + z_4)$$

该定义下的算子称之为Prewitt算子：

-1	-1	-1
0	0	0
1	1	1

水平

-1	0	1
-1	0	1
-1	0	1

垂直

0	1	1
-1	0	1
-1	-1	0

对角线

-1	-1	0
-1	0	1
0	1	1

Sobel算子

Sobel算子是在Prewitt算子的基础上改进的，在中心系数上使用一个权值2，相比较Prewitt算子，Sobel模板能够较好的抑制（平滑）噪声。

计算公式为：

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$
$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Sobel算子：

-1	-2	-1
0	0	0
1	2	1

水平

-1	0	1
-2	0	2
-1	0	1

垂直

0	1	2
-1	0	1
-2	-1	0

对角线

-2	-1	0
-1	0	1
0	1	2

Laplacian算子

大部分算子（如Sobel或Scharr）都是下面减上面或者右边减左边。实际上他们都是计算的一次，也就是下面减上面一次，右边减左边一次。而Laplacian算子计算了两次，也就是上减中，下减中，左减中，右减中。所以Laplacian算子相当于二阶Sobel算子的导数。

Laplacian算子在灰度恒定区域其结果为0，在像素均匀变化区域其结果也为0，在像素变化速率变化较大区域其值也会变大，因此可将拉普拉斯算子应用到图像锐化和边缘检测。

Laplacian算子：

0	1	0
1	-4	1
0	1	0

例子：

P1	P2	P3
P4	P5	P6
P7	P8	P9

$$P5_{\text{new}} = (p2 + p4 + p6 + p8) - 4 * p5$$

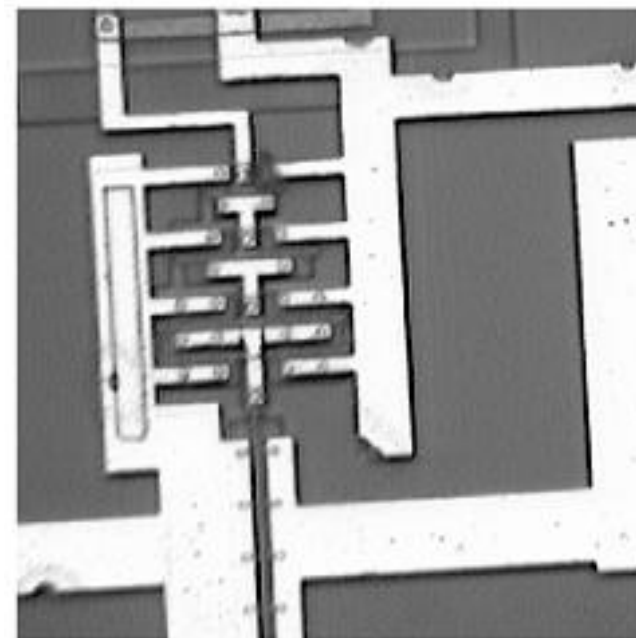
图像模糊

例:

```
ic=imread('ic.tif');  
px=[-1 0 1;-1 0 1;-1 0 1];  
icx=filter2(px,ic);  
figure,imshow(icx/255)  
py=px';  
icy=filter2(py,ic);  
figure,imshow(icy/255)  
edge_p=sqrt(icx.^2+icy.^2);  
figure, imshow(edge_p/255)  
edge_t=im2bw(edge_p/255,0.3);
```

We can obtain edges by the Prewitt filters directly by using the command

```
edge_p=edge(ic,'prewitt');
```





Function `edge` find edges in intensity image.

- `EDGE` takes an intensity image `I` as its input, and returns a binary image `BW` of the same size as `I`, with 1's where the function finds edges in `I` and 0's elsewhere.
- `EDGE` supports six different edge-finding methods:
 - The Sobel method
 - The Prewitt method
 - The Roberts method
 - The Laplacian of Gaussian method
 - The zero-cross method
 - The Canny method

例:

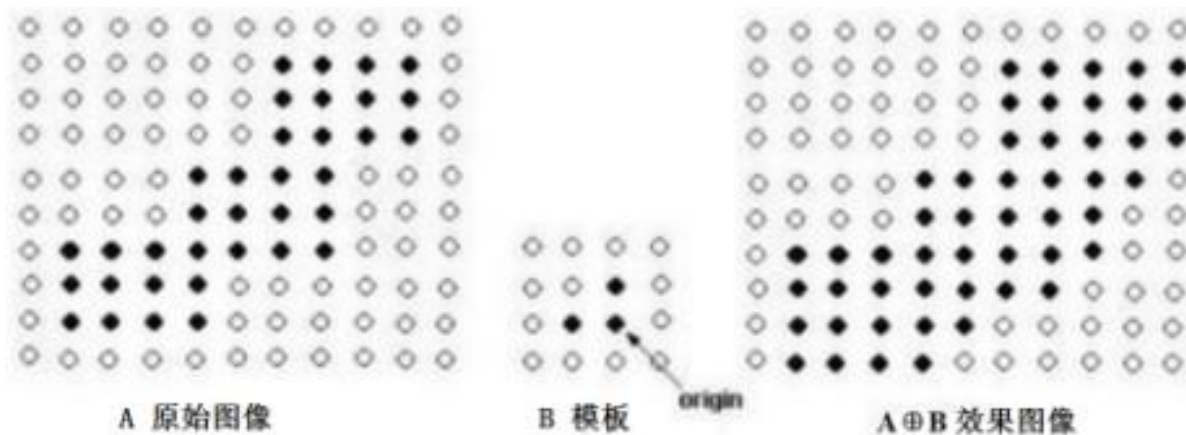
```
I=imread('Will.jpg');  
I=rgb2gray(I);  
BW1=edge(I,'sobel');  
BW2=edge(I,'prewitt');  
BW3=edge(I,'roberts');
```



膨胀 (Dilation)

膨胀的运算符是“ $\oplus$ ”，其定义为：
$$A \oplus B = \{x \mid (B)_x \cap A \neq \emptyset\}$$

该公式表示用B来对图像A进行膨胀处理，其中B是一个卷积模板或卷积核，其形状可以为正方形或圆形，通过模板B与图像A进行卷积计算，扫描图像中的每一个像素点，用模板元素与二值图像元素做“与”运算，如果都为0，那么目标像素点为0，否则为1。从而计算B覆盖区域的像素点最大值，并用该值替换参考点的像素值实现膨胀。

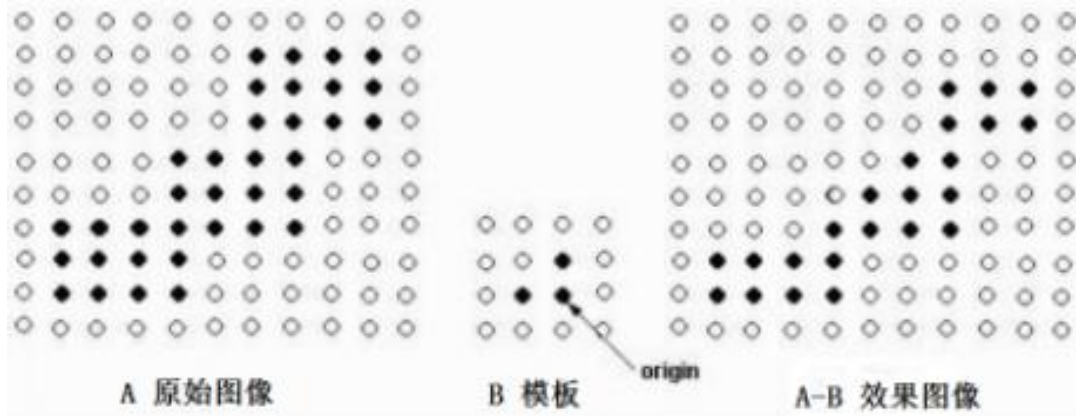


上图是将左边的原始图像A膨胀处理为右边的效果图 $A \oplus B$

腐蚀 (Erosion)

腐蚀的运算符是“ $-$ ”，其定义为： $A - B = \{x \mid B_x \subseteq A\}$

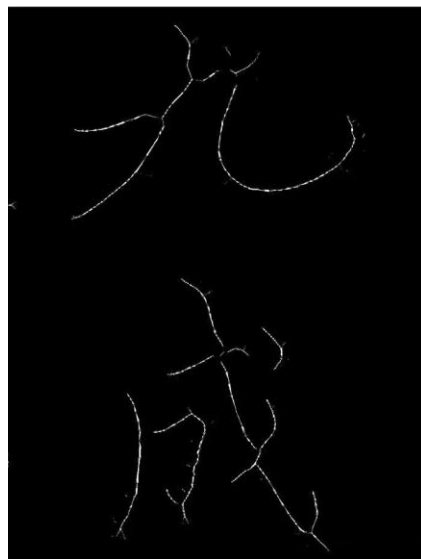
该公式表示图像A用卷积模板B来进行腐蚀处理，通过模板B与图像A进行卷积计算，得出B覆盖区域的像素点最小值，并用这个最小值来替代参考点的像素值。



如上图所示，将左边的原始图像A腐蚀处理为右边的效果图A-B。

骨架 (Skeleton)

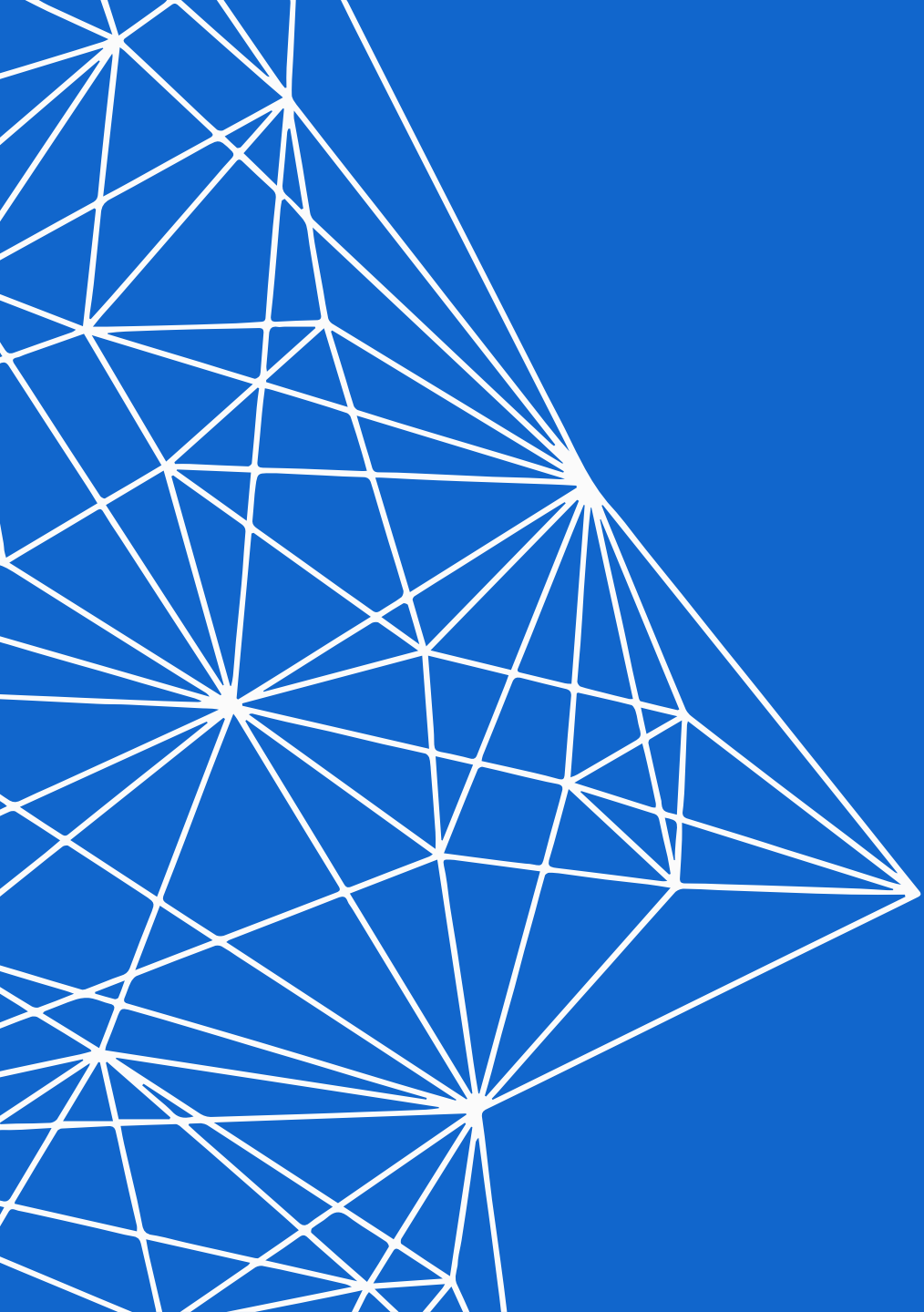
对于腐蚀的扩展



骨架提取，也叫二值图像细化。这种算法能将一个连通区域细化成一个像素的宽度，用于特征提取和目标拓扑表示。骨架提取与分水岭算法也属于形态学处理范畴，都放在morphology子模块内。

典型骨架模型：

1. Max-Disc
2. Fire



THANKS!